

**Lab Assignment 6**  
**Object-Oriented Programming**

**Branch**  
**B.E., 2<sup>nd</sup> Year – ENC**



**Submitted By:**  
**Kaushtubh Kumar**  
**Roll No. 1024150191**  
**Batch: 2023**

## Question 1

Implement a class Library that contains an array of Book objects. The Book class should have attributes like title, author, and ISBN. The Library class should provide methods: addNewBook, removeBooks, displayDetails.

### Code

```
#include <iostream>
#include <string>
using namespace std;

class Book {
public:
    string title;
    string author;
    string ISBN;
};

class Library {
private:
    Book books[100];
    int bookCount;
public:
    Library() : bookCount(0) {}

    bool addNewBook(string &title, string &author, string &ISBN) {
        if (bookCount >= 100) return false;
        books[bookCount].title = title;
        books[bookCount].author = author;
        books[bookCount].ISBN = ISBN;
        bookCount++;
        return true;
    }

    bool removeBooks(string &ISBN) {
        for (int i = 0; i < bookCount; i++) {
            if (books[i].ISBN == ISBN) {
                for (int j = i; j < bookCount - 1; j++) {
                    books[j] = books[j + 1];
                }
                bookCount--;
                return true;
            }
        }
        return false;
    }

    void displayDetails() {
        cout << "=== Library Books ===" << endl;
        for (int i = 0; i < bookCount; i++) {
            cout << "Book " << i + 1 << ":" << endl;
            cout << "  Title: " << books[i].title << endl;
            cout << "  Author: " << books[i].author << endl;
        }
    }
};
```

```

        cout << "   ISBN: " << books[i].ISBN << endl;
    }
}

};

int main() {
    Library lib;

    string t1 = "The Great Gatsby", a1 = "F. Scott Fitzgerald", i1 = "978-0743273565";
    string t2 = "1984", a2 = "George Orwell", i2 = "978-0451524935";
    string t3 = "To Kill a Mockingbird", a3 = "Harper Lee", i3 = "978-0061120084";
    string t4 = "Pride and Prejudice", a4 = "Jane Austen", i4 = "978-0141439518";
    string t5 = "The Catcher in the Rye", a5 = "J.D. Salinger", i5 = "978-0316769488";

    lib.addNewBook(t1, a1, i1);
    lib.addNewBook(t2, a2, i2);
    lib.addNewBook(t3, a3, i3);
    lib.addNewBook(t4, a4, i4);
    lib.addNewBook(t5, a5, i5);

    lib.displayDetails();

    cout << "\n--- Removing Book ---" << endl;
    lib.removeBooks(i2);
    lib.displayDetails();

    return 0;
}

```

=== Library Books ===

Book 1:

Title: The Great Gatsby  
Author: F. Scott Fitzgerald  
ISBN: 978-0743273565

Book 2:

Title: 1984  
Author: George Orwell  
ISBN: 978-0451524935

Book 3:

Title: To Kill a Mockingbird  
Author: Harper Lee  
ISBN: 978-0061120084

Book 4:

Title: Pride and Prejudice  
Author: Jane Austen  
ISBN: 978-0141439518

Book 5:

Title: The Catcher in the Rye  
Author: J.D. Salinger  
ISBN: 978-0316769488

--- Removing Book ---

=== Library Books ===

Book 1:

Title: The Great Gatsby  
Author: F. Scott Fitzgerald  
ISBN: 978-0743273565

Book 2:

Title: To Kill a Mockingbird  
Author: Harper Lee  
ISBN: 978-0061120084

Book 3:

Title: Pride and Prejudice  
Author: Jane Austen  
ISBN: 978-0141439518

Book 4:

## Question 2

Implement Question 1 using constructors. Add default constructor, parameterized constructor, and copy constructor. Use the this pointer instead of scope resolution operator.

### Code

```
#include <iostream>
#include <string>
using namespace std;

class Book {
public:
    string title;
    string author;
    string ISBN;

    Book() {
        this->title = "";
        this->author = "";
        this->ISBN = "";
    }

    Book(string title, string author, string ISBN) {
        this->title = title;
        this->author = author;
        this->ISBN = ISBN;
    }

    Book(const Book &other) {
        this->title = other.title;
        this->author = other.author;
        this->ISBN = other.ISBN;
    }
};

class Library {
private:
    Book books[100];
    int bookCount;
public:
    Library() : bookCount(0) {}

    bool addNewBook(string &title, string &author, string &ISBN) {
        if (bookCount >= 100) return false;
        books[bookCount] = Book(title, author, ISBN);
        bookCount++;
        return true;
    }

    bool removeBooks(string &ISBN) {
        for (int i = 0; i < bookCount; i++) {
            if (books[i].ISBN == ISBN) {
```

```

        for (int j = i; j < bookCount - 1; j++) {
            books[j] = books[j + 1];
        }
        bookCount--;
        return true;
    }
    return false;
}

void displayDetails() {
    cout << "=== Library Books ===" << endl;
    for (int i = 0; i < bookCount; i++) {
        cout << "Book " << i + 1 << ":" << endl;
        cout << "  Title: " << books[i].title << endl;
        cout << "  Author: " << books[i].author << endl;
        cout << "  ISBN: " << books[i].ISBN << endl;
    }
}

};

int main() {
    // Array of objects using Initializer list
    Book bookList[] = {
        Book("The Great Gatsby", "F. Scott Fitzgerald", "978-0743273565"),
        Book("1984", "George Orwell", "978-0451524935"),
        Book("To Kill a Mockingbird", "Harper Lee", "978-0061120084")
    };

    // Dynamic initialization
    Book* dynamicBooks = new Book[2];
    dynamicBooks[0] = Book("Pride and Prejudice", "Jane Austen", "978-0141439518");
    dynamicBooks[1] = Book("The Catcher in the Rye", "J.D. Salinger", "978-0316769488");

    Library lib;
    string t1 = "The Great Gatsby", a1 = "F. Scott Fitzgerald", i1 = "978-0743273565";
    string t2 = "1984", a2 = "George Orwell", i2 = "978-0451524935";
    string t3 = "To Kill a Mockingbird", a3 = "Harper Lee", i3 = "978-0061120084";
    string t4 = "Pride and Prejudice", a4 = "Jane Austen", i4 = "978-0141439518";
    string t5 = "The Catcher in the Rye", a5 = "J.D. Salinger", i5 = "978-0316769488";

    lib.addNewBook(t1, a1, i1);
    lib.addNewBook(t2, a2, i2);
    lib.addNewBook(t3, a3, i3);
    lib.addNewBook(t4, a4, i4);
    lib.addNewBook(t5, a5, i5);

    lib.displayDetails();
}

```

```
    delete[] dynamicBooks;  
    return 0;  
}
```

```
=== Library Books ===
```

```
Book 1:
```

```
    Title: The Great Gatsby  
    Author: F. Scott Fitzgerald  
    ISBN: 978-0743273565
```

```
Book 2:
```

```
    Title: 1984  
    Author: George Orwell  
    ISBN: 978-0451524935
```

```
Book 3:
```

```
    Title: To Kill a Mockingbird  
    Author: Harper Lee  
    ISBN: 978-0061120084
```

```
Book 4:
```

```
    Title: Pride and Prejudice  
    Author: Jane Austen  
    ISBN: 978-0141439518
```

```
Book 5:
```

```
    Title: The Catcher in the Rye  
    Author: J.D. Salinger  
    ISBN: 978-0316769488
```

```
...Program finished with exit code 0
```

```
Press ENTER to exit console
```

### Question 3

Create a class Account with attributes account number (const long), transaction ID, transaction type, and balance. Implement depositAmount, creditAmount, and displayDetails functions.

#### Code

```
#include <iostream>
#include <string>
using namespace std;

class Account {
private:
    const long accountNumber;
    long transactionId;
    string transactionType;
    double balance;
    static long nextTransactionId;

public:
    Account(long accNum, double initialBalance)
        : accountNumber(accNum), balance(initialBalance), transactionId(0),
        transactionType("None") {}

    long depositAmount(const long &to, const long &from, const double &amount)
    {
        transactionId = nextTransactionId++;
        transactionType = "Deposit";
        balance += amount;
        cout << "Deposited $" << amount << " to account " << to << endl;
        return transactionId;
    }

    long creditAmount(const long &to, const long &from, const double &amount) {
        transactionId = nextTransactionId++;
        transactionType = "Credit";
        if (balance >= amount) {
            balance -= amount;
            cout << "Credited $" << amount << " from account " << from << endl;
            return transactionId;
        }
        cout << "Insufficient balance!" << endl;
        return -1;
    }

    void displayDetails() const {
        cout << "=== Account Details ===" << endl;
        cout << "Account Number: " << accountNumber << endl;
        cout << "Balance: $" << balance << endl;
        cout << "Last Transaction ID: " << transactionId << endl;
        cout << "Last Transaction Type: " << transactionType << endl;
    }
};
```



```

long Account::nextTransactionId = 1000;

int main() {
    Account acc1(1234567890, 5000.0);
    Account acc2(9876543210, 3000.0);

    acc1.depositAmount(acc1.accountNumber, 0, 1000.0);
    acc2.depositAmount(acc2.accountNumber, 0, 500.0);

    acc1.creditAmount(acc2.accountNumber, acc1.accountNumber, 200.0);

    cout << endl;
    acc1.displayDetails();
    cout << endl;
    acc2.displayDetails();

    return 0;
}

```

```

Deposited $1000 to account 1234567890
Deposited $500 to account 9876543210
Transferred $200 from account 1234567890 to account 9876543210

=== Account Details ===
Account Number: 1234567890
Balance: $5800
Last Transaction ID: 1002
Last Transaction Type: Transfer

=== Account Details ===
Account Number: 9876543210
Balance: $3700
Last Transaction ID: 1001
Last Transaction Type: Deposit

...Program finished with exit code 0
Press ENTER to exit console.

```

## Question 4

Write a program to add data objects of two different classes using friend functions.

### Code

```
#include <iostream>
using namespace std;

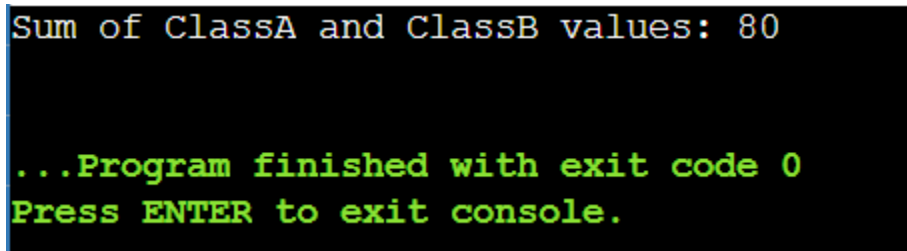
class ClassB; // Forward declaration

class ClassA {
private:
    int valueA;
public:
    ClassA(int v) : valueA(v) {}
    friend void addObjects(ClassA &a, ClassB &b);
};

class ClassB {
private:
    int valueB;
public:
    ClassB(int v) : valueB(v) {}
    friend void addObjects(ClassA &a, ClassB &b);
};

void addObjects(ClassA &a, ClassB &b) {
    int sum = a.valueA + b.valueB;
    cout << "Sum of ClassA and ClassB values: " << sum << endl;
}

int main() {
    ClassA objA(50);
    ClassB objB(30);
    addObjects(objA, objB);
    return 0;
}
```

A screenshot of a console window with a black background. The first line of output is "Sum of ClassA and ClassB values: 80" in a light blue/cyan monospaced font. The second line is "...Program finished with exit code 0" in a bright green monospaced font. The third line is "Press ENTER to exit console." in the same bright green monospaced font.

Sum of ClassA and ClassB values: 80

...Program finished with exit code 0  
Press ENTER to exit console.

## Question 5

Define a class named Complex with properties (real and imaginary) and methods: Parameterized constructor, copy constructor, display(), and sum() as friend function.

### Code

```
#include <iostream>
using namespace std;

class Complex {
private:
    double real;
    double imaginary;
public:
    Complex(double r = 0, double i = 0) : real(r), imaginary(i) {}

    Complex(const Complex &other) {
        this->real = other.real;
        this->imaginary = other.imaginary;
    }

    void display() {
        cout << real << " + " << imaginary << "i" << endl;
    }

    friend Complex sum(Complex &c1, Complex &c2);
};

Complex sum(Complex &c1, Complex &c2) {
    Complex result;
    result.real = c1.real + c2.real;
    result.imaginary = c1.imaginary + c2.imaginary;
    return result;
}

int main() {
    Complex c1(3.5, 4.2);
    Complex c2(1.5, 2.8);

    cout << "Complex Number 1: ";
    c1.display();

    cout << "Complex Number 2: ";
    c2.display();

    Complex c3 = sum(c1, c2);
    cout << "Sum: ";
    c3.display();

    Complex c4(c1); // Copy constructor
    cout << "Copy of c1: ";
    c4.display();
}
```

```
    return 0;  
}
```

Complex Number 1:  $3.5 + 4.2i$

Complex Number 2:  $1.5 + 2.8i$

Sum:  $5 + 7i$

Copy of c1:  $3.5 + 4.2i$

...Program finished with exit code 0

Press ENTER to exit console.